Implementation of Basic Communication in ROS 2 Using Python

Preksha Samajpati

Department of Computer Science and Engineering

Manipal Institute of Technology, MAHE

Manipal, India

preksha.mitmpl2026@learner.manipal.edu

***Abstract*—This report reviews the core concepts of the Robot Operating System 2 (ROS 2): the computation graph and its building blocks (nodes, topics, services, actions and parameters), the difference between topics and services, custom interfaces, the colcon build tool with workspace structure, and Quality of Service (QoS) settings. Two Python implementations written with rclpy on ROS 2 Humble are then explained with their ROS graphs: a publisher–subscriber pair that computes the longitudinal speed of a wheel from a custom message, and a three-node pipeline that checks whether a user-supplied string is a palindrome.**

***Keywords* —ROS 2, rclpy, computation graph, topics, services, actions, parameters, custom messages, colcon, QoS.**

I. INTRODUCTION

Modern robots combine perception, planning, control, and actuation components that are often developed by different teams, written in different languages, and deployed across several computers. ROS is a software suite that allows for quick and easy building of autonomous robotic systems. The Robot Operating System (ROS) is a set of software libraries and tools for building robot applications. From drivers and state-of-the-art algorithms to powerful developer tools, ROS has the open-source tools you need for your next robotics project.

Section II to Section VI cover the concepts requested in the assignment. Sections VII and VIII describe the two practical solutions, each with its ROS graph.

II. ROS 2 COMPUTATION GRAPH

The *Computation Graph* is the peer-to-peer network of ROS processes that are processing data together. The basic Computation Graph concepts of ROS are *nodes, topics, services, actions, parameters* all of which provide data to the Graph in different ways.

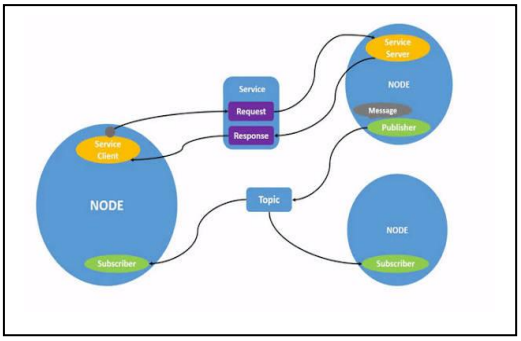


Fig 1: Visual Depiction of ROS 2 Computation Graph

A. Nodes

A node is the basic unit of computation. Nodes are processes that perform computation. ROS is designed to be modular at a fine-grained scale; a robot control system usually comprises many nodes. For example, one node controls a laser range-finder, one node controls the wheel motors, one node performs localization, one node performs path planning, one node provides a graphical view of the system, and so on.

B. Topics

Messages are routed via a transport system with publish / subscribe semantics. A node sends out a message by *publishing* it to a given topic. The topic is a name that is used to identify the content of the message. A node that is interested in a certain kind of data will *subscribe* to the appropriate topic. There may be multiple concurrent publishers and subscribers for a single topic, and a single node may publish and/or subscribe to multiple topics. In general, publishers and subscribers are not aware of each other's' existence. The idea is to decouple the production of information from its consumption. Logically, one can think of a topic as a strongly typed message bus. Each bus has a name, and anyone can connect to the bus to send or receive messages as long as they are the right type.

C. Services

The publish / subscribe model is a very flexible communication paradigm, but its many-to-many, one-way transport is not appropriate for request /

reply interactions, which are often required in a distributed system. Request / reply is done via services, which are defined by a pair of message structures: one for the request and one for the reply. A providing node offers a service under a name and a client uses the service by sending the request message and awaiting the reply.

D. Actions

Actions handle long-running tasks that need progress reports and can be cancelled, for example "navigate to this pose". An action type has three parts: a goal, a result and periodic feedback. An action client sends a goal, receives feedback while the server works, may cancel, and finally receives the result. Internally an action is built from topics and services: three services (send goal, cancel goal, get result) and two topics (feedback and status).

E. Parameters

Parameters are typed configuration values owned by a node, such as a wheel radius, a gain or a frame name. A node declares a parameter with declare parameter, giving a name, a type and a default.

II. TOPICS VERSUS SERVICES

TABLE I

COMPARISON OF TOPICS AND SERVICES

Aspect	**Topic**	**Service**
Pattern	Publish/subscribe, many-to-many	Request/response, one server, many clients
Timing	Continuous or event driven, asynchronous	On demand, one call at a time
Reply	None; publisher does not know who received	Explicit response to each call

Use a topic when data is produced repeatedly and several consumers may need it, or when the producer should not wait for anyone. Examples are camera frames, wheel speeds and velocity commands. A late subscriber simply sees the next message.

Use a service when the caller needs an answer for one specific request, the operation is short, and it happens occasionally, for example "is this string a palindrome?" asked once, or "save the map". If the task takes long or needs progress feedback, use an **action** instead.

III. CUSTOM INTERFACES

ROS 2 describes data with interface files. A .msg file defines a message for topics, a .srv file defines a request and a response separated by ---, and a .action file defines goal, result and feedback separated by ---. Fields are built from primitive types (float32, int32, string, bool), arrays, or other messages. Type file names use CamelCase (for example TaskOne.msg), and field names must be lowercase with underscores, which is why angVel is written ang_vel in this report.

Interfaces are generated by rosidl. The package must be an ament_cmake package, since Humble does not generate interfaces in pure Python packages. The required pieces are:

- CMakeLists.txt: find_package(rosidl_default_generators REQUIRED) and rosidl_generate_interfaces(${PROJECT_NAME} "msg/Name.msg").
- package.xml: buildtool_depend on rosidl_default_generators, exec_depend on rosidl_default_runtime, and member_of_group rosidl_interface_packages.
- Users import the generated type, for example from wheel_speed.msg import TaskOne, after building and sourcing the workspace.

Larger projects usually keep interfaces in a separate package so that other packages can depend on them. In both solutions here the message and the nodes sit in one ament_cmake package to keep the build simple.

IV. COLCON AND WORKSPACE STRUCTURE

A **ROS 2 workspace** is a dedicated directory used to organize, build, and modify ROS 2 packages. It manages the build boundaries of your project and keeps your custom code safely isolated from the core ROS 2 installation. Colcon is the tool that builds them. A package declares its name, version and dependencies in package.xml, and its build type (ament_cmake or ament_python) in the export section. The standard layout is:

```
taskone/              (workspace root)
 src/                 source packages
  wheel_speed/
    msg/TaskOne.msg
    scripts/publisher.py
    CMakeLists.txt
    package.xml
 build/               intermediate files
 install/             built packages, setup.bash
 log/                 build logs
```

Only src/ is source code, so only it is placed under version control; build/, install/ and log/ are regenerated. The usual workflow is colcon build from the workspace root, then source install/setup.bash in every new terminal. --packages-select builds one package, and --symlink-install lets edited Python files take effect without rebuilding.

V. QUALITY OF SERVICE

QoS profiles may be configured for publishers and subscriptions independently. A connection between a publisher and a subscription is only made if the pair has compatible QoS profiles. The most important ones are listed in Table II.

TABLE II

MAIN QOS POLICIES

Policy	Options	Effect
Reliability	Reliable, Best effort	Reliable retries lost samples; best effort may drop them for lower latency
Durability	Volatile, Transient local	Transient local lets late subscribers receive the last stored samples
History / Depth	Keep last N, Keep all	How many samples are queued for a slow reader
Deadline	Duration	Maximum time allowed between samples
Lifespan	Duration	Age after which a sample is discarded
Liveliness	Automatic, Manual; lease	Detects that a publisher is still alive

A publisher and subscriber connect only if their policies are *compatible*: the publisher must offer at least what the subscriber requests. A best-effort publisher cannot serve a reliable subscriber, and a volatile publisher cannot serve a transient-local subscriber. Mismatches produce a warning and no data.

VI. SOLUTION 1: WHEEL SPEED

A. Problem and design

The task is to publish the angular velocity and radius of a wheel typed by the user, and to compute the longitudinal speed in a subscriber. For a wheel rolling without slip the speed is $v = \omega \cdot r$, with ω in rad/s and r in metres. The package wheel_speed contains the message TaskOne and two nodes, publisher and subscriber, linked by the topic /taskone.

```
# msg/TaskOne.msg
float32 ang_vel
float32 radius
```

B. Implementation

The publisher reads two numbers from the terminal in a loop, fills a TaskOne message and publishes it on /taskone. The subscriber registers a callback that multiplies the two fields and prints the result:

```
#scripts/Publisher.py
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from wheel_speed.msg import TaskOne
rclpy.init()
node = Node('publisher')
pub = node.create_publisher(TaskOne, '/taskone', 10)
while rclpy.ok():
    msg = TaskOne()
    msg.ang_vel = float(input('Angular velocity (rad/s): '))
    msg.radius = float(input('Radius (m): '))
    pub.publish(msg)
    print('Published')
#scripts/Subscriber.py
#!/usr/bin/env python3
```

```
import rclpy
from rclpy.node import Node
from wheel_speed.msg import TaskOne
rclpy.init()
node = Node('subscriber')
def callback(msg):
    speed = msg.ang_vel * msg.radius
    print(f'Longitudinal speed: {speed:.3f} m/s')
node.create_subscription(TaskOne, '/taskone', callback, 10)
rclpy.spin(node)
```

Both scripts are installed with install(PROGRAMS ...) in CMakeLists.txt and started with ros2 run wheel_speed subscriber.py and ros2 run wheel_speed publisher.py.

C. Result and ROS Graph:

Entering ω = 10 rad/s and r = 0.25 m gives the subscriber output Longitudinal speed: 2.500 m/s.



Fig 2: Publisher asking for inputs



Fig 3: Subscriber giving the outputs

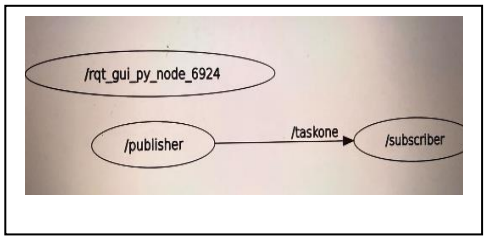


Fig 4: rqt graph of solution 1

VII. SOLUTION 2: PALINDROME CHECKER

A. Problem and design

Three nodes in the package rt2_task cooperate. Node 1 publishes a user-typed string repeatedly using the custom message RT2String. Node 2 checks whether the string is a palindrome and sends 1 or 0 to Node 3. Node 3 answers Yes or No, and Node 2 prints the answer.

msg/RT2String.msg

```
string data
```

B. Implementation:

Node 1 runs a 1 Hz timer that publishes the current string, while a background thread waits for new input, so that the blocking input() call does not stop the node from spinning. Typing a new string simply replaces the one being published.

Node 2 subscribes to /rt2_string, compares the string with its reverse and publishes the integer result. It also subscribes to /answer and prints the reply. **Node 3** converts the integer into text.

```
#!/usr/bin/env python3
import threading
import rclpy
from rclpy.node import Node
from rt2_task.msg import RT2String
rclpy.init()
node = Node('node1')
pub = node.create_publisher(RT2String, '/rt2_string', 10)
current = RT2String()
def publish_current():
    if current.data != '':
        pub.publish(current)
node.create_timer(1.0, publish_current)
def get_input():
    while True:
        current.data = input('Enter a string: ')
threading.Thread(target=get_input, daemon=True).start()
try:
```

```
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass


    #!/usr/bin/env python3
    import rclpy
    from rclpy.node import Node
    from std_msgs.msg import Int32, String
    from rt2_task.msg import RT2String
    rclpy.init()
    node = Node('node2')
    pub = node.create_publisher(Int32,
'/is_palindrome', 10)
    last = {'text': ''}
    def on_string(msg):
        last['text'] = msg.data
        result = Int32()
        result.data = 1 if msg.data == msg.data[::-1]
else 0
        pub.publish(result)
    def on_answer(msg):
        print(f"{last['text']} -> {msg.data}")
    node.create_subscription(RT2String, '/rt2_string',
on_string, 10)
    node.create_subscription(String, '/answer',
on_answer, 10)
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass


#!/usr/bin/env python3

import rclpy

from rclpy.node import Node

from std_msgs.msg import Int32, String

rclpy.init()

node = Node('node3')

pub = node.create_publisher(String, '/answer', 10)

def on_value(msg):

    reply = String()

    reply.data = 'Yes' if msg.data == 1 else 'No'

    pub.publish(reply)

node.create_subscription(Int32, '/is_palindrome',
on_value, 10)

try:

    rclpy.spin(node)

except KeyboardInterrupt:

    pass
```

C. Result and ROS Graph:

The comparison is case sensitive. Table IV shows sample inputs, and Fig. 3 shows the graph, in which Node 2 both feeds Node 3 and receives its reply.

TABLE III

SAMPLE TEST CASES

Input	**/is_palindrome**	**/answer**
AABAA	1	Yes
BACAB	1	Yes
ABC	0	No
Aba	0	No

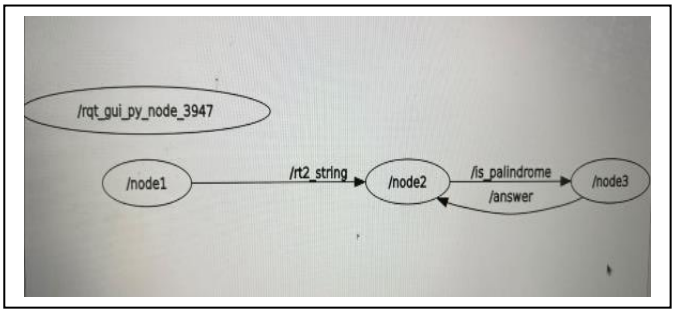


Fig 5: rqt graph of solution 2

VIII. CONCLUSION

This task gave me a much more practical understanding of ROS 2 than I had going in, not just the commands, but why the pieces exist and how they fit together. Building the wheel-speed and palindrome packages showed me how nodes, topics and custom messages form a computation graph. Seeing that graph appear in rqt_graph, with my own nodes connected by the topics I defined, made the idea of anonymous publish/subscribe communication much clearer than reading about it did.

IX. ACKNOWLEDGMENT

I would like to thank the Driverless Subsystem seniors for providing the guidance and resources that helped me complete the task with a deeper understanding of ROS 2 and the concepts discussed in this report.

X. REFERENCES

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, Art. no. eabm6074, 2022.

[2] Open Robotics, "ROS 2 Documentation: Humble Hawksbill."[Online].Available: https://docs.ros.org/en/humble.

[3] Object Management Group, *Data Distribution Service (DDS)*, Version 1.4, 2015.

[4] "colcon: collective construction." [Online]. Available: https://colcon.readthedocs.io/.